

torch.compiler.nested_compile_region#

https://pytorch.org/docs/stable/generated/torch.compiler.nested_compile_region.html

Description:

Tells `torch.compile` that the marked set of operations forms a nested compile region (which is often repeated in the full model) whose code can be compiled once and safely reused. `nested_compile_region` can also be used as a decorator. During `torch.compile` tracing, the compiler applies hierarchical compilation with `nested_compile_region`: it emits optimized code for the marked region the first time it is encountered and re-emits (or “stamps out”) the previously compiled code on every subsequent invocation. This can substantially reduce overall compile time for deeply-stacked, structurally-identical components such as the transformer layers of a large-language-model (LLM). Outside a `torch.compile` context—i.e., in standard eager execution—the call is a no-op, so existing workflows remain unaffected. Note that `nested_compile_region` does not promise that a region will be compiled exactly once. If the compiler detects that new input conditions (shape, dtype, device, stride, globals etc.) make the cached version invalid to reuse, it will transparently re-compile the region. Using it is therefore safe: correctness is always preserved, and you pay the extra compilation cost only

Function Signature

```
torch.compiler.nested_compile_region(fn=None)[source]#
```

Detailed Documentation

Documentation

Tells `torch.compile` that the marked set of operations forms a nested compile region (which is often repeated in the full model) whose code can be compiled once and safely reused. `nested_compile_region` can also be used as a decorator.

During `torch.compile` tracing, the compiler applies hierarchical compilation with `nested_compile_region`: it emits optimized code for the marked region the first time it is encountered and re-emits (or “stamps out”) the previously compiled code on every subsequent invocation. This can substantially reduce overall compile time for deeply-stacked, structurally-identical components such as the transformer layers of a large-language-model (LLM).

Outside a `torch.compile` context—i.e., in standard eager execution—the call is a no-op, so existing workflows remain unaffected.

Note that `nested_compile_region` does not promise that a region will be compiled exactly once. If the compiler detects that new input conditions (shape, dtype, device, stride, globals etc.) make the cached version invalid to reuse, it will transparently re-compile the region. Using it is therefore safe: correctness is always preserved, and you pay the extra compilation cost only when required.